Do Time-Series Foundation Models Need Fine-Tuning? Wind Power Forecasting with PatchTST and Chronos

DATA612 PWS1 Group Project, Interim Report

Simran Kharbanda Tanmay Sharma
UID: 122283671 UID: 121997234

October 29, 2026

1 Introduction and problem statement

Wind power is difficult to forecast because the output follows wind speed, which varies from minute to minute. Grid operators rely on short-term forecasts to schedule reserves and balance supply with demand, and forecast errors are costly in both directions: over-forecasting leaves a shortfall that must be covered at spot prices, and under-forecasting means reserve capacity is bought and not used.

A team building such a forecaster today has three options: train a small transformer on the farm's own history, use a pretrained time-series foundation model as it is, or fine-tune that foundation model on the farm's data. Which of these performs best, and at what cost in data and GPU time, is an open practical question, and it is the one this project tests.

The task is as follows: for each of the 134 turbines in the SDWPF wind farm [4], take the last $L = 288$ ten-minute readings of active power (two days) and forecast the next H readings, for $H = 6$, 36 and 64 (one hour, six hours, and about eleven hours). We compare PatchTST [1], a transformer we implement ourselves and train from scratch, against Chronos [2], a pretrained T5 forecaster, used three ways: zero-shot, fully fine-tuned, and fine-tuned with LoRA [3]. We stop at $H = 64$ because that is the longest horizon Chronos was trained for; going further would hamper it for reasons that have nothing to do with our questions.

2 Objectives and scope

The three research questions from the proposal are unchanged.

RQ1 Does PatchTST trained from scratch on SDWPF beat zero-shot Chronos?

RQ2 How much of the gap, in either direction, do full fine-tuning and LoRA close?

RQ3 How does each method behave with 5%, 25% and 100% of the training data, and what does it cost in GPU hours and trainable parameters?

The work we have committed to is the full grid: {PatchTST, Chronos full fine-tune, Chronos LoRA} × {5%, 25%, 100% of the training data} × 3 seeds, plus zero-shot Chronos and three baselines (persistence, seasonal naive, LSTM), with every model scored by the same evaluation code on the same windows. We also set ourselves a gate before any of this runs on wind data: our PatchTST had to reproduce the paper's ETTh1 numbers first. That gate has been passed (Section 4).

Two things are optional enhancements: a multi-GPU fine-tune of the larger Chronos-T5-base, and a spatial extension that gives PatchTST its neighbouring turbines as extra inputs. We are not using covariates such as wind speed as model inputs (Chronos is univariate and has no way to take

them, so the main comparison stays univariate for all models.), not going beyond 64 steps, and not using the extended two-year SDWPF release.

3 Methodology

Data and splits. SDWPF [4] covers 134 turbines over 245 days at ten-minute resolution: 4,727,520 rows, ten sensor columns, and the target `Patv` in kW. We follow the dataset paper's cleaning rules. Negative power is clamped to zero. A reading is *unknown* when power is at or below zero while the wind is above 2.5 m/s, or when any blade pitch angle exceeds 89°; it is *abnormal* when the nacelle direction is outside ±720° or the wind direction outside ±180°; and it is *missing* when `Patv` is NaN. Flagged readings stay in the model inputs as real history, but they never count towards the loss or the metrics.

The split is chronological and aligned to whole days: 171 days of training, 25 of validation, 49 of test. The 5% and 25% subsets are the most recent 1,231 and 6,156 steps of the training period, nested, so a model trained on 25% has seen everything the 5% model saw. Forecast origins in validation and test sit every 36 steps, at 00:00, 06:00, 12:00 and 18:00, which gives 99 validation and 195 test origins per turbine, identical for every model. Training windows use stride 1 and must lie entirely inside their subset. At evaluation time the history may reach back into the previous period, but a target never crosses a split boundary.

PatchTST. We implemented it from the paper in plain PyTorch and used the official repository only to check details we were unsure about. The pieces are per-channel instance normalisation (RevIN, without the affine parameters), patches of length 16 with stride 8, a linear patch embedding with a learned positional encoding, a post-norm transformer encoder that uses BatchNorm as the official code does, and a flatten-plus-linear head for each channel. Channels are independent: each variable passes through the same weights as its own batch element. The quantile head for probabilistic forecasts comes next (PT-02).

Chronos. We use Chronos-T5-small (46M parameters) from the original Chronos release, chosen because its training-data list is published and does not include SDWPF. Zero-shot forecasts come straight from the `chronos-forecasting` package. Full fine-tuning uses the repository's own `scripts/training/train.py`, fed with our GluonTS Arrow export. LoRA goes on the T5 attention projections through `peft`, which we add ourselves because the repository has no LoRA support. All three predict 64 steps, and we read the shorter horizons off the same forecast.

Baselines and metrics. The baselines are persistence, seasonal naive (the same time yesterday, $m = 144$) and a single-layer LSTM. All metrics are computed over valid readings only: MAE and RMSE in kW, MASE with a seasonal-naive denominator computed on the full training period for every model, and weighted quantile loss over $q \in \{0.1, \ldots, 0.9\}$ for the models that produce quantiles. On the cost side we record GPU hours as Slurm service units, trainable parameters, and inference time.

Tools and compute. Everything is PyTorch 2.5.1, with Hugging Face `transformers`, `peft`, `gluonts` and `chronos-forecasting` for the Chronos side, `pyarrow` and `pandas` for the data, and `pytest` for the tests (64 at the moment). Training runs on UMD's Zaratan cluster through Slurm. Most jobs fit on an A100 MIG slice (5 GB, 7 SU per hour); we only take a full A100 (48 SU per hour) when memory leaves no choice.

4 Progress so far

The project is organised as 24 GitHub issues in five phases, one branch and one pull request per issue. The base-paper phase (BP) is finished and merged. The data phase (DS) has three of its

Table 1: Work items. "Done" means merged to `main`; data items were reviewed by the other member first.

Item	Work	Owner	Status
BP-01	Zaratan environment, Slurm smoke test on a MIG slice	Simran	Done
BP-02	ETTh1 loading, 12/4/4-month split, train-fitted scaler, sliding windows	Simran	Done
BP-03	RevIN, patching, channel merge/split, with unit tests	Simran	Done
BP-04	PatchTST model matching the official code (81,728 parameters at $H = 96$)	Simran	Done
BP-05	Training loop with the official recipe, best-validation checkpoint, test once	Simran	Done
BP-06	ETTh1 $H = 96$ run: matches the paper (Table 2)	Simran	Done
BP-08	$H = 192/336/720$ and the paper's three ablations	Simran	Done
DS-01	SDWPF download with checksums, cleaning rules, validity mask, dense arrays	Tanmay	Done
DS-02	Day-aligned split, nested subsets, fixed evaluation origins, coverage report	Tanmay	Done
DS-03	GluonTS Arrow export for Chronos training, verified with Chronos's own `train.py`	Tanmay	Done
DS-04	Shared evaluation module	Tanmay	Open
DS-05–07	Naive baselines, LSTM, Chronos zero-shot	Tanmay	Open
PT-01–03	PatchTST on SDWPF, quantile head, head-design pilot	Simran	Open
CF-01–03	Chronos full fine-tune, LoRA, DDP on T5-base	Tanmay / Simran	Open
EX-01–03	Slurm job array, run the grid, analysis	Tanmay / Simran	Open

seven items merged. Table 1 lists everything with its current status.

PatchTST reproduces the paper. In the proposal we promised to get within 5% of the paper's ETTh1 error before touching wind data. We are within 0.5% at every horizon (Table 2). The setting is the paper's PatchTST/42: $L = 336$, patch length 16 with stride 8 (42 patches), three encoder layers, model width 16, four heads, feed-forward width 128, dropout 0.3, Adam at 10^{-4}, batch size 128, seed 2021, 100 epochs, and the checkpoint with the lowest validation MSE is tested once. For comparability, note that the official code drops the last incomplete test batch, so at $H = 96$ it scores 2,688 of the 2,785 test windows. We report that number alongside the full test set. Each run takes about seven minutes on a MIG slice, under 1 SU.

Table 2: ETTh1 test error (MSE / MAE), our implementation against the paper's Table 3, and the paper's Table 6 ablations at $H = 96$. "Official" is the drop-last scoring; "full" is all test windows.

Setting	H	Paper	Ours (official)	Ours (full)	Params	Train
PatchTST/42	96	0.375 / 0.399	0.374 / 0.398	0.378 / 0.399	81,728	7 min
	192	0.414 / 0.421	0.414 / 0.421	0.414 / 0.421	146,336	7 min
	336	0.431 / 0.436	0.431 / 0.436	0.441 / 0.441	243,248	7 min
	720	0.449 / 0.466	0.447 / 0.464	0.455 / 0.470	501,680	6 min
No RevIN	96	–	0.387 / 0.411	0.391 / 0.412	81,728	7 min
Channel mixing	96	–	0.415 / 0.437	0.419 / 0.438	470,912	1.4 min
No patching ($p = s = 1$)	96	–	0.365 / 0.396	0.368 / 0.397	539,328	19 min (full A100)

Ablations. These reproduce the paper's reasoning, not just its headline number. Taking RevIN out costs 3% in MSE. Mixing channels costs 11% and needs six times the parameters. Removing patching altogether, one token per time step, is 2% more accurate, but it has 6.6× the parameters and 64× the attention memory, and it would not fit on a MIG slice at all. Patching buys cost, not accuracy, which is how the paper frames it too. The whole base-paper phase used about 22 SU.

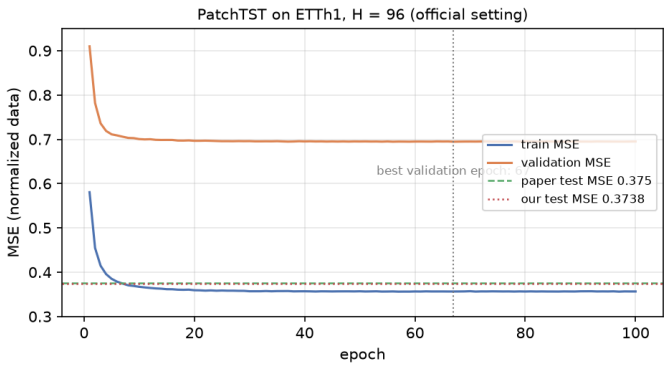


Figure 1: ETTh1 $H = 96$: training and validation MSE per epoch. The best validation epoch (67) is the checkpoint that is tested; the test MSE of 0.374 is the paper's 0.375.

Data pipeline is built and audited. DS-01 downloads SDWPF from the Hugging Face mirror with SHA-256 checks and applies the cleaning rules. Of the 4,727,520 readings, 3,595,859 (76%) are valid. Another 1,082,113 (23%) are unknown, almost all of it curtailment, where the turbine reports zero power although the wind is above cut-in speed; 49,518 are missing and 127 abnormal. 1,263,062 negative readings were clamped. DS-02 fixes the split and, because it counts valid readings per window, revealed several problems: day 66 is a farm-wide outage with no valid readings at all, validation day 193 and test days 229 to 233 are under 25% valid, and the worst turbine is 67% valid over the whole period. At $H = 6$, one in five validation windows has no valid target. So the evaluation keeps every origin and scores per reading with a mask instead of dropping windows. The origin lists and per-window counts are committed, which is what guarantees every model sees the same windows. DS-03 writes each training subset and the validation period as GluonTS Arrow files, with a switch for whether flagged readings are kept or blanked, and we confirmed on Zaratan that Chronos's unmodified `train.py` loads and trains on all eight files.

5 Challenges and changes

Problems encountered.

- **Memory on MIG slices.** The no-patching ablation needs about 1.5 GB per layer just for attention at 337 tokens and batch 128, and the 5 GB slice ran out. Rather than change the code with gradient accumulation and lose comparability, we ran it once on a full A100 (19 minutes, about 15 SU).
- **Queue waits.** A full A100 took several hours to come through. We now keep everything that fits on MIG slices and submit long jobs early with checkpoint-resume.
- **The Chronos environment.** `transformers` 5.x fails to import on our `torch` 2.5.1, so it is pinned below 5. Chronos's extras pull in a newer `pyarrow`, so that pin became a minimum. And `train.py` only accepts options through a YAML file, which we now keep in the repository.

Changes from the proposal.

- **Flagged readings in inputs.** The proposal said we would gap-fill inputs by carrying the last value forward. We now keep unknown and abnormal readings in the inputs as real values, because the turbine did produce zero when it was curtailed, and we mask them only in the loss and the metrics. Chronos cannot make that distinction, since its training script treats a NaN as both a missing input and a missing target. So DS-03 exports both variants and CF-01 will run both on the 5% subset and report the difference.
- **One model per horizon or one for all.** The paper trains a separate PatchTST per horizon.

Chronos predicts 64 steps and we read off 6 and 36. To keep the comparison fair, PT-01 will train PatchTST the same way, one $H = 64$ model, and we will note the per-horizon variant as the paper's practice.

- **What "validation" means.** Early stopping and model selection use the 99 shared origins per turbine, not every stride-1 window. That way the validation numbers we report are the same ones that picked the model.

6 Remaining work and timeline

Table 3: Plan to the final report. Dates are targets; the grid gate is the one that matters.

Dates	Work	Owner	Items
Oct 30 – Nov 7	Shared evaluation module; persistence and seasonal-naive baselines	Tanmay	DS-04, DS-05
Oct 30 – Nov 10	PatchTST on SDWPF with masked loss; quantile head; head-design pilot at full data	Simran	PT-01–03
Nov 8 – Nov 14	LSTM baseline; Chronos zero-shot on the fixed windows	Tanmay	DS-06, DS-07
Nov 10 – Nov 21	Chronos full fine-tune (both flagged-reading variants on 5%, then the grid); LoRA via `peft`	Tanmay / Simran	CF-01, CF-02
Nov 17 – Nov 24	Slurm job array; run the 27-run grid with 3 seeds	both	EX-01, EX-02
Nov 24	**Gate:** grid finished, all results JSON committed	Tanmay / Simran	
Nov 21 – Nov 28	Optional: DDP fine-tune of Chronos-T5-base on 2 A100s, if the pool allows	Simran	CF-03
Nov 25 – Dec 2	Analysis and plots: error vs. data fraction, cost vs. accuracy, per-horizon tables	Tanmay / Simran	EX-03
Dec 3 / Dec 10	Presentation; final report due Dec 10	both	

On compute, we expect the grid to cost about 200 SU: PatchTST runs are under 1 SU each and a Chronos-small fine-tune is a few SU on a MIG slice. The optional T5-base runs would add up to 100 SU. We have used about 22 SU so far.

7 Individual contributions

Simran Kharbanda. Set up the repository, the issue and PR conventions, and the Zaratan environment and Slurm scripts (BP-01). Implemented and verified PatchTST end to end, BP-02 through BP-06, then the three remaining horizons and the three ablations, including the memory workaround for no-patching (BP-08). Checked the Chronos environment on Zaratan (version pins, the `train.py` smoke config) for DS-01 to DS-03. Next: PT-01 to PT-03, LoRA fine-tuning (CF-02), the DDP run (CF-03), the Slurm job array (EX-01), and the shared grid and analysis work.

Tanmay Sharma. Built the data pipeline: SDWPF download, cleaning and the validity mask with per-turbine and per-day audits (DS-01); the splits, nested subsets, fixed evaluation origins and the window-coverage report (DS-02); and the GluonTS Arrow export with the flagged-reading switch and validation file (DS-03). Next: the shared evaluation code (DS-04), the baselines and the LSTM (DS-05, DS-06), Chronos zero-shot (DS-07), full fine-tuning (CF-01), the analysis (EX-03), and the shared grid work.

References

[1] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. *ICLR*, 2023. Code: https://github.com/yuqinie98/PatchTST

[2] A. F. Ansari et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. https://arxiv.org/abs/2403.07815

[3] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.

[4] J. Zhou et al. SDWPF: A dataset for spatial dynamic wind power forecasting over a large turbine array. *Scientific Data*, 11, 2024. https://arxiv.org/abs/2208.04360

[5] T. Kim et al. Reversible instance normalization for accurate time-series forecasting against distribution shift. *ICLR*, 2022.

[6] H. Zhou et al. Informer: Beyond efficient transformer for long sequence time-series forecasting (ETTh1 dataset). *AAAI*, 2021.

[7] Amazon Science. chronos-forecasting (software, Apache-2.0). https://github.com/amazon-science/chronos-forecasting

[8] Hugging Face. PEFT: Parameter-efficient fine-tuning (software). https://github.com/huggingface/peft